

STEP 1 — Create Project Folder

Create folder:

banking-agent

Open this folder in VS Code.

STEP 2 — Open Terminal in VS Code

Menu:

Terminal → New Terminal

STEP 3 — Create Virtual Environment

Run:

python -m venv .venv

STEP 4 — Activate Virtual Environment

Windows PowerShell

Run:

.venv\Scripts\activate

You should now see:

(.venv)

in terminal.

STEP 5 — Install Required Packages

Run:

pip install fastapi uvicorn langgraph langchain openai azure-search-documents python-dotenv python-multipart

STEP 6 — Create Files

Create these files inside project folder:

main.py

workflow.py

.env

STEP 7 — Add Banking Policy Files

Add your policy files:

Loan Policy.txt

Fraud Policy.txt

Compliance Policy.txt

Credit Card Policy.txt

STEP 9 — Upload Documents into Blob Storage

Create Storage Account

Search:

Storage Account

Create new storage account.

Create Container

Inside storage account:

Containers → + Container

Container name:

bankagent-data

Access level:

Private

Upload Files

Open container.

Click:

Upload

Upload all .txt files.

STEP 10 — Import Data into Azure AI Search

Open:

Azure AI Search

Click:

Import Data

STEP 11 — Select Data Source

Choose:

Azure Blob Storage

STEP 12 — Select Scenario

Choose:

RAG

STEP 13 — Configure Blob Storage

Fill:

Field	Value
--------------	--------------

Storage Account	your storage
------------------------	---------------------

Blob Container	bankagent-data
-----------------------	-----------------------

Field	Value
Parsing Mode	Default

Then click:

Next

STEP 14 — Vectorize Text

Choose:

Field	Value
Kind	Microsoft Foundry
Project	Your Foundry Project
Model Deployment	text-embedding-3-small

Check:

I acknowledge...

Then:

Next

STEP 15 — Skip Image Enrichment

Click:

Next

STEP 16 — Advanced Settings

Keep default.

Click:

Next then go to search explorer tab and wait for few time

STEP 17 — check its automaticaly Create Index under our ai services → search management in left panel → indexes (eg. Rag-423..)

STEP 18 — Test Search

Open:

Search Explorer on ai search page

Search:

loan approval

You should see policy results.

STEP 20 — Get Azure OpenAI / Foundry Details

STEP 21 — Add .env File

Paste:

AZURE_OPENAI_API_KEY=your_openai_key

AZURE_OPENAI_ENDPOINT=https://your-openai-endpoint.openai.azure.com/

AZURE_OPENAI_DEPLOYMENT=gpt-4o-mini

AZURE_SEARCH_ENDPOINT=https://your-search-name.search.windows.net

AZURE_SEARCH_KEY=your_search_admin_key

AZURE_SEARCH_INDEX=rag-xxxxxxx

STEP 22 — Create workflow.py

Paste your LangGraph workflow code.

```
from typing import TypedDict
```

```
from langgraph.graph import StateGraph, END
```

```
from azure.search.documents import SearchClient
```

```
from azure.core.credentials import AzureKeyCredential
```

```
from dotenv import load_dotenv
```

```
from openai import AzureOpenAI
```

```
import os
```

```
# -----
```

```
# LOAD ENV VARIABLES
```

```
# -----
```

```
load_dotenv()
```

```
# -----
```

```
# AZURE OPENAI
```

```
# -----
```

```
client = AzureOpenAI(
```

```
    api_key=os.getenv("AZURE_OPENAI_API_KEY"),
```

```
    api_version="2024-02-15-preview",
```

```
    azure_endpoint=os.getenv("AZURE_OPENAI_ENDPOINT")
```

)

deployment = os.getenv("AZURE_OPENAI_DEPLOYMENT")

AZURE AI SEARCH

```
search_client = SearchClient(
    endpoint=os.getenv("AZURE_SEARCH_ENDPOINT"),
    index_name=os.getenv("AZURE_SEARCH_INDEX"),
    credential=AzureKeyCredential(
        os.getenv("AZURE_SEARCH_KEY")
    )
)
```

STATE

class BankingState(TypedDict):

user_input: str

retrieved_context: str

final_result: str

```
# RETRIEVAL NODE
```

```
# -----
```

```
def retrieval_node(state):
```

```
    query = state["user_input"]
```

```
    results = search_client.search(
```

```
        search_text=query,
```

```
        top=3
```

```
    )
```

```
    context = ""
```

```
    for result in results:
```

```
        if "chunk" in result:
```

```
            context += result["chunk"] + "\n\n"
```

```
        elif "content" in result:
```

```
            context += result["content"] + "\n\n"
```

```
    return {
```

```
        "retrieved_context": context
```

```
    }
```



```
# -----
```

```
# ANALYSIS NODE
```

```
# -----
```

```
def analysis_node(state):
```

```
    prompt = f"""
```

```
You are an Enterprise Banking AI Assistant.
```

```
Answer ONLY using the banking policy context below.
```

```
If information is unavailable,
```

```
say:
```

```
"Information not available in banking policies."
```

```
=====
```

```
BANKING POLICY CONTEXT
```

```
=====
```

```
{state['retrieved_context']}
```

```
=====
```

```
USER QUESTION
```

```
=====
```

```
{state['user_input']}
```

```
"""
```

```
    response = client.chat.completions.create(
```

```
model=deployment,
```

```
messages=[
```

```
{
```

```
  "role": "system",
```

```
  "content": "You are a professional banking AI assistant."
```

```
},
```

```
{
```

```
  "role": "user",
```

```
  "content": prompt
```

```
}
```

```
],
```

```
temperature=0.3
```

```
)
```

```
answer = response.choices[0].message.content
```

```
return {
```

```
  "final_result": answer
```

```
}
```

```
# -----
```

```
# LANGGRAPH WORKFLOW
```

```
# -----
```

```
workflow = StateGraph(BankingState)
```

```
workflow.add_node(  
    "retrieval",  
    retrieval_node  
)
```

```
workflow.add_node(  
    "analysis",  
    analysis_node  
)
```

```
workflow.set_entry_point(  
    "retrieval"  
)
```

```
workflow.add_edge(  
    "retrieval",  
    "analysis"  
)
```

```
workflow.add_edge(  
    "analysis",  
    END  
)
```

```
# -----
```

```
# COMPILE GRAPH
```

```
# -----
```

```
graph = workflow.compile()
```

STEP 23 — Create main.py

Paste FastAPI web application code.

```
from fastapi import FastAPI, Form
```

```
from fastapi.responses import HTMLResponse
```

```
from workflow import graph
```

```
app = FastAPI()
```

```
# -----
```

```
# HOME PAGE
```

```
# -----
```

```
@app.get("/", response_class=HTMLResponse)
```

```
async def home():
```

```
    return """
```

```
<html>
```

```
<head>
```

```
    <title>Enterprise Banking AI Assistant</title>
```

```
</head>
```

```
<body style="
```

```
font-family: Arial;

background-color: #f4f4f4;

padding: 40px;

">
```

```
<div style="

background: white;

padding: 30px;

border-radius: 10px;

max-width: 900px;

margin: auto;

box-shadow: 0px 0px 10px rgba(0,0,0,0.1);

">
```

```
<h1>🏦 Enterprise Banking AI Assistant</h1>
```

```
<p>

Ask questions related to:

loans, fraud detection, compliance,

KYC policies, and banking rules.

</p>
```

```
<form action="/analyze" method="post">
```

```
<textarea

name="user_input"

rows="8"

style="

width: 100%;
```

```
padding: 15px;
font-size: 16px;
border-radius: 6px;
"
placeholder="Ask banking question here..."
></textarea>
```

```
<br><br>
```

```
<button
type="submit"
style="
background-color: #0078D4;
color: white;
padding: 12px 20px;
border: none;
border-radius: 5px;
font-size: 16px;
cursor: pointer;
"
>
Analyze
</button>
```

```
</form>
```

```
</div>
```

```
</body>
```

```
</html>
```

```
"""
```

```
# -----
```

```
# ANALYZE ROUTE
```

```
# -----
```

```
@app.post("/analyze", response_class=HTMLResponse)
```

```
async def analyze(user_input: str = Form(...)):
```

```
    result = graph.invoke({
```

```
        "user_input": user_input
```

```
    })
```

```
    final_answer = result["final_result"]
```

```
    return f'"""
```

```
<html>
```

```
<head>
```

```
    <title>AI Banking Decision</title>
```

```
</head>
```

```
<body style="
```

```
    font-family: Arial;
```

```
    background-color: #f4f4f4;
```

```
    padding: 40px;
```

```
<div style="
  background: white;
  padding: 30px;
  border-radius: 10px;
  max-width: 1000px;
  margin: auto;
  box-shadow: 0px 0px 10px rgba(0,0,0,0.1);
">
```

```
<h1>🏠 AI Banking Decision</h1>
```

```
<h3>User Question</h3>
```

```
<div style="
  background-color: #f0f0f0;
  padding: 15px;
  border-radius: 8px;
  margin-bottom: 25px;
  white-space: pre-wrap;
  line-height: 1.7;
">
```

```
{user_input}
```

```
</div>
```

```
<h3>AI Response</h3>
```



```
<div style="
    background-color: #eef6ff;
    padding: 20px;
    border-radius: 8px;
    white-space: pre-wrap;
    line-height: 1.9;
    font-size: 16px;
">
```

```
{final_answer}
```

```
</div>
```

```
<br><br>
```

```
<a href="/"
    style="
        text-decoration: none;
        color: #0078D4;
        font-weight: bold;
">
```

```
← Ask Another Question
```

```
</a>
```

```
</div>
```

```
</body>
```

```
</html>
```

```
""""
```

STEP 24 — Run Application

Run:

uvicorn main:app --reload

STEP 25 — Open Browser

Open:

http://127.0.0.1:8000

STEP 26 — Test Questions

Ask:

What documents are required for loan approval?

What are fraud detection rules?

What are KYC compliance requirements?

FINAL ARCHITECTURE

User Question



FastAPI Web App



LangGraph Workflow



Azure AI Search (RAG Retrieval)



Relevant Banking Policies



Azure OpenAI / GPT-4o-mini



Final AI Response